Hand Gesture Virtual Mouse

Controlling Computer Input Through Real-Time Hand Tracking

Course: Image and Video Processing

Students: Jash Jain (H020) | Yashi Saxena (H007) | Udayan Singh Champawat (H026)

Programme: MBA Tech | 3 Year | Semester V

Faculty: Dr. Raj Gaurav Mishra

Institute: NMIMS / STME

Academic Year: 2026 - 2027

Abstract

This project presents a hand gesture-based virtual mouse system that enables touchless computer interaction using a standard webcam. The problem addressed is the dependency on physical input devices in situations where hands-free or contactless control is desirable, such as sterile environments, accessibility scenarios, and public kiosks.

The proposed solution uses real-time hand tracking via MediaPipe to detect 21 skeletal landmarks per hand, classifies gestures through angle-based finger extension analysis and scale-invariant pinch detection, and maps recognized gestures to operating-system-level mouse events using PyAutoGUI. Key image and video processing techniques employed include live frame capture with OpenCV, colour-space conversion (BGR to RGB), landmark-based feature extraction, adaptive signal filtering using the 1-Euro algorithm, and hysteresis-driven state classification.

The system supports cursor movement, left click, right click, double click, drag-and-drop, scrolling, and screenshot capture, all driven by distinct single-hand gestures. Testing under typical indoor conditions yielded frame rates of 19-29 FPS with reliable gesture recognition and stable cursor behaviour, demonstrating that practical touchless input is achievable with commodity hardware and open-source computer vision libraries.

1. Introduction

1.1 Background

Human-computer interaction has traditionally relied on physical peripherals such as the mouse, keyboard, and touchscreen. While these devices are effective for most use cases, there are contexts where physical contact with input hardware is impractical or undesirable. Surgical environments demand sterility, industrial settings may involve hazardous materials, and users with motor impairments may find conventional devices difficult to operate. These scenarios motivate the development of touchless interaction methods that leverage computer vision to interpret natural human gestures as input commands.

1.2 Relevance of the Problem

The global market for gesture recognition technology continues to grow, driven by applications in healthcare, automotive interfaces, smart home systems, and augmented reality. While dedicated hardware solutions like the Leap Motion controller offer precise tracking, they require specialized sensors and additional cost. A software-only approach using a standard webcam lowers the barrier to entry significantly and makes touchless interaction accessible on any modern laptop or desktop computer without hardware modification.

1.3 Role of Image and Video Processing

Image and video processing form the foundation of this system. Each webcam frame undergoes colour-space conversion from BGR to RGB before being passed to the MediaPipe Hands model, which performs real-time detection and localization of 21 hand keypoints. These landmarks serve as the raw features from which gesture states are derived through geometric analysis of joint angles and inter-finger distances. Temporal smoothing via adaptive digital filtering ensures that the extracted pointer trajectory is both stable during stationary positioning and responsive during active movement. Without these processing stages, raw landmark data would be too noisy for reliable cursor control.

1.4 Motivation

This project was selected to explore the intersection of computer vision and human-computer interaction within the scope of an image and video processing course. It provides a concrete application of several core topics covered in the curriculum, including real-time video processing, feature extraction from visual data, signal filtering, and pattern classification. Additionally, building a complete end-to-end system from webcam input to OS-level output offers valuable experience in integrating multiple processing stages into a coherent pipeline.

2. Problem Statement and Objectives

2.1 Problem Statement

Conventional computer input requires physical contact with hardware devices. In environments where touchless interaction is preferred or necessary, there is a need for a low-cost, software-based alternative that can translate natural hand gestures into standard mouse operations using only a webcam, without requiring specialized sensors or hardware modifications.

2.2 Objectives

1. To implement a real-time hand tracking pipeline using MediaPipe that reliably detects and tracks 21 hand landmarks from a standard webcam feed at interactive frame rates.
2. To develop a gesture classification system based on finger-joint angle analysis and scale-invariant pinch detection that distinguishes between move, click, drag, right-click, double-click, scroll, and screenshot gestures.
3. To apply adaptive signal filtering (1-Euro filter) to the extracted pointer trajectory, minimizing cursor jitter during stationary positioning while maintaining responsiveness during active hand movement.
4. To map classified gestures to operating-system-level mouse events (cursor movement, clicks, drag-and-drop, scrolling, screenshots) with appropriate debouncing, cooldowns, and state management to prevent accidental triggers.
5. To evaluate the system's performance under typical indoor conditions in terms of frame rate, gesture recognition accuracy, and cursor stability, and to document its limitations and areas for future improvement.

3. Scope of the Project

3.1 What the Project Covers

- Real-time single-hand gesture recognition from a standard webcam feed.
- Seven distinct gesture types mapped to mouse operations: cursor move, left click, drag-and-drop, right click, double click, scroll, and screenshot.
- Adaptive cursor smoothing using the 1-Euro filter algorithm.
- Visual feedback overlay on the camera feed showing hand skeleton, gesture state, FPS counter, and action notifications.
- Debug mode displaying live finger angles and pinch distance ratios for tuning.
- Cross-platform compatibility (Windows, macOS, Linux).

3.2 What the Project Does Not Cover - Two-handed or multi-hand gesture recognition.

- Body pose tracking or facial expression analysis.
- GPU-accelerated inference or custom-trained neural network models.
- Integration with specific application APIs or remote device control.
- Hardware design or sensor fabrication.

3.3 Type of Input Used

The system operates exclusively on live webcam video captured at 640x480 resolution. No pre-recorded dataset is used; all processing occurs in real time on the incoming video stream. Screenshots captured during operation serve as validation artifacts and are stored in PNG format.

3.4 Application Domain

The primary application domains include touchless human-computer interaction for sterile or hazardous environments, assistive technology for users with limited motor function, public kiosk interfaces, and prototyping for AR/VR interaction paradigms.

4. Dataset / Input Description

This project does not use a pre-existing dataset. All input is acquired in real time from a standard webcam. The following table summarizes the input characteristics:

Parameter	Value
Source	Built-in / USB webcam
Resolution	640 x 480 pixels
Frame Format	BGR (OpenCV default), converted to RGB for MediaPipe
Capture Backend	DirectShow (Win) / AVFoundation (macOS) / V4L2 (Linux)
Frame Rate	19 - 29 FPS (observed, CPU-dependent)
Hand Model	MediaPipe Hands, 21 landmarks, world + screen coords
Detection Confidence	0.7 (both detection and tracking)
Model Complexity	Level 1 (full model)
Output Artifacts	PNG screenshots saved to ./Screenshots directory

Sample input frames captured during testing are included in the Results section below. These frames show the live camera feed with overlaid hand skeleton landmarks, gesture state indicators, and the active tracking region boundary.

5. Methodology / Proposed Approach

The system follows a sequential processing pipeline applied to each incoming webcam frame. The main stages are described below.

5.1 Processing Pipeline

Stage 1 - Frame Capture

Webcam frame acquired via OpenCV at 640x480, mirrored horizontally for natural view.

Stage 2 - Landmark Detection

Frame converted BGR to RGB, passed to MediaPipe Hands model. Outputs 21 screen-space landmarks (normalized 0-1) and 21 world-space 3D landmarks (metric coordinates).

Stage 3 - Landmark Smoothing

World landmarks smoothed via exponential moving average (alpha = 0.6) to reduce high-frequency detector noise while preserving deliberate movements.

Stage 4 - Gesture Classification

Finger extension determined by PIP/DIP joint angles with hysteresis (150 deg extend, 135 deg fold). Pinch detected via thumb-to-fingertip distance normalized by palm size, with enter/exit thresholds (0.35 / 0.55). Gesture debounce over 2 consecutive frames.

Stage 5 - Cursor Control

Index knuckle (landmark 5) mapped to screen coordinates. 1-Euro filter applied independently to X and Y axes.
Offset-decay mechanism prevents jumps after lock periods.

Stage 6 - Action Dispatch

Gesture transitions trigger mouse events via PyAutoGUI. Drag initiated after 0.4s hold. Cooldown of 0.6s between discrete actions. Post-click freeze of 0.2s.

Stage 7 - Visual Feedback

Hand skeleton, gesture label, FPS counter, and action notifications drawn onto the camera frame and displayed in an OpenCV window.

5.2 Algorithm Overview

The gesture classifier maintains internal state across frames. Finger extension flags use hysteresis to avoid flickering near decision boundaries. Pinch detection tracks the closest eligible finger (one not folded below 110 degrees) and applies separate enter and exit thresholds to create a stable activation zone. The gesture state machine requires a new gesture to persist for a configurable number of consecutive frames before being accepted, rejecting transient misdetections. When no hand is detected for 8 consecutive frames, all active states including drag, scroll anchors, and filter history are fully reset.

6. Techniques / Algorithms Used

6.1 Image Processing Techniques

Colour-Space Conversion

Each BGR frame from OpenCV is converted to RGB before MediaPipe inference, as the model expects RGB input. This is performed using cv2.cvtColor with the COLOR_BGR2RGB flag.

Feature Extraction (Landmark Detection)

MediaPipe Hands extracts 21 keypoints per hand, providing both screen-space (normalized) and world-space (metric 3D) coordinates. Joint angles are computed from triplets of world landmarks using the dot-product formula: angle = arccos((v1 . v2) / (|v1| * |v2|)).

Temporal Smoothing (Exponential Moving Average)

Raw world landmarks are smoothed each frame using EMA with alpha = 0.6: smoothed = 0.6 * raw + 0.4 * previous. This reduces high-frequency jitter from the detector while preserving responsiveness to deliberate hand movements.

Adaptive Signal Filtering (1-Euro Filter)

The 1-Euro filter is an adaptive low-pass filter whose cutoff frequency increases with estimated signal velocity. At low speeds, aggressive smoothing eliminates jitter; at high speeds, reduced smoothing minimizes phase lag. Parameters: min_cutoff = 1.2 Hz, beta = 12.0, d_cutoff = 1.0 Hz.

Hysteresis Thresholding

Both finger extension detection and pinch activation use dual thresholds with hysteresis. Finger extension requires 150 degrees to activate and drops below 135 degrees to deactivate. Pinch enters at ratio < 0.35 and exits at ratio > 0.55. This prevents rapid state toggling when values hover near a single threshold.

6.2 Video Processing Techniques

Real-Time Frame Capture

Continuous video acquisition at 640x480 using OpenCV VideoCapture. On Windows, the DirectShow backend is attempted first for faster initialization. Each frame is horizontally mirrored to provide a natural mirror-view experience.

Object Tracking (Hand Skeletal Model)

MediaPipe Hands provides continuous per-frame tracking of a single hand with 21 connected keypoints forming a skeletal model. Tracking confidence threshold of 0.7 ensures reliable association across frames. Loss of tracking for 8 consecutive frames triggers a full reset.

Gesture State Machine with Debouncing

A finite state machine manages gesture transitions. New gestures must persist for 2 consecutive frames before acceptance (debounce). Action-specific cooldowns (0.6s) prevent rapid re-triggering. Drag detection uses a temporal threshold (0.4s hold) to distinguish intentional drags from quick clicks.

7. Implementation

7.1 Technology Stack

Component	Technology	Purpose
Language	Python 3.x	Primary implementation language
Video Capture	OpenCV (cv2)	Webcam access, frame display, overlay drawing
Hand Tracking	MediaPipe Hands v0.10.14	21-keypoint skeletal model
Mouse Control	PyAutoGUI	Cross-platform cursor, clicks, scroll, screenshots
Numerics	NumPy (< 2.0)	Vector operations on landmark arrays

7.2 Hardware Requirements

- Standard webcam (built-in or USB)
- CPU capable of running MediaPipe inference at 20+ FPS (tested on Apple M-series and Intel i5+)
- No GPU required; all processing runs on CPU
- Minimum 4 GB RAM recommended

7.3 Key Implementation Steps

1. Webcam initialization with fallback backend selection (DirectShow on Windows, default elsewhere).
2. MediaPipe Hands configured with model_complexity=1, max_num_hands=1, and confidence thresholds of 0.7 for both detection and tracking.
3. World landmarks extracted as a 21x3 NumPy float32 array and smoothed via EMA each frame.
4. GestureClassifier computes per-finger joint angles and thumb-to-finger distance ratios each frame, applying hysteresis and eligibility checks.
5. HandMouse controller maintains cursor state, applies 1-Euro filtering, manages gesture transitions with debouncing, and dispatches PyAutoGUI actions.
6. HUD overlay drawn each frame using cv2.putText and cv2.circle/line for skeleton visualization.
7. Keyboard handler processes P (pause), D (debug toggle), and Q/ESC (quit) inputs.

8. Results and Discussion

The system was tested across multiple sessions under typical indoor lighting conditions. The following observations summarize the key results.

8.1 Performance Metrics

Metric	Observed Value

Frame Rate	19 - 29 FPS
Detection Confidence Threshold	0.7
Cursor Jitter (stationary hand)	< 2 px after filtering
Click Recognition Rate	High under consistent hand positioning
Drag Activation Threshold	0.4 seconds hold
Action Cooldown	0.6 seconds
Hand Loss Reset Delay	8 frames (~0.3s at 25 FPS)

8.2 Sample Output Frames

The following screenshots were captured during live testing sessions. Each frame shows the camera feed with overlaid hand skeleton landmarks (red dots connected by white lines), the active tracking region (magenta rectangle), current gesture mode (top-left), FPS counter (top-right), and action feedback (bottom).

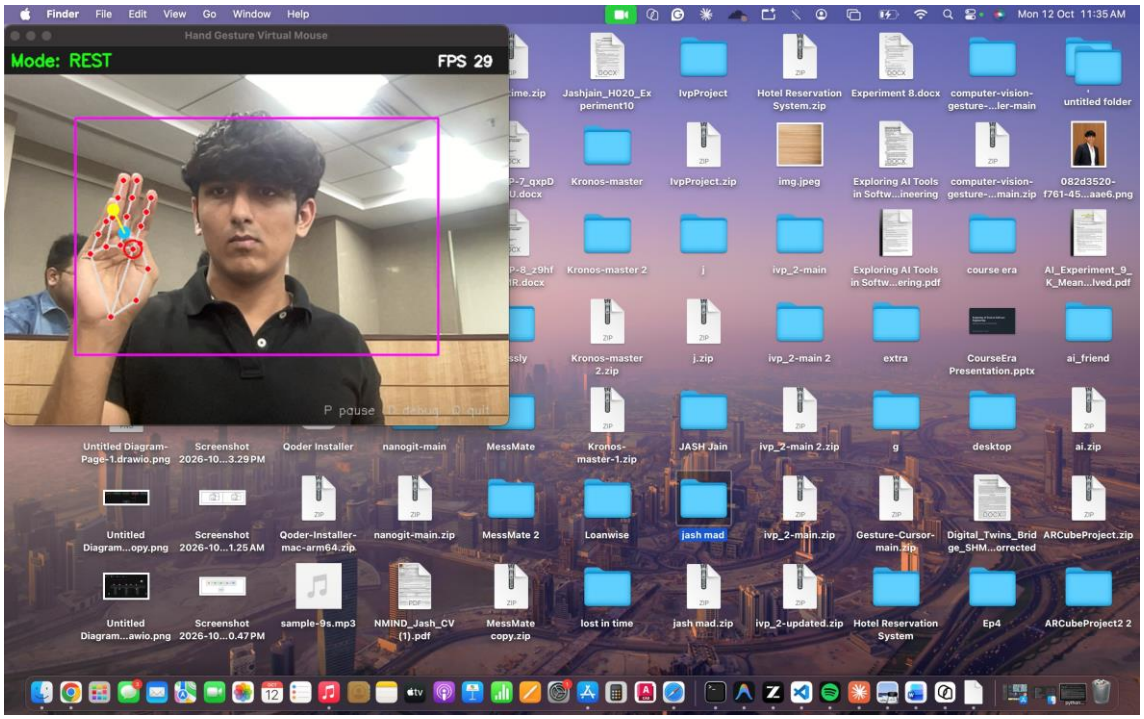


Figure 1: System in REST mode with open palm detected.

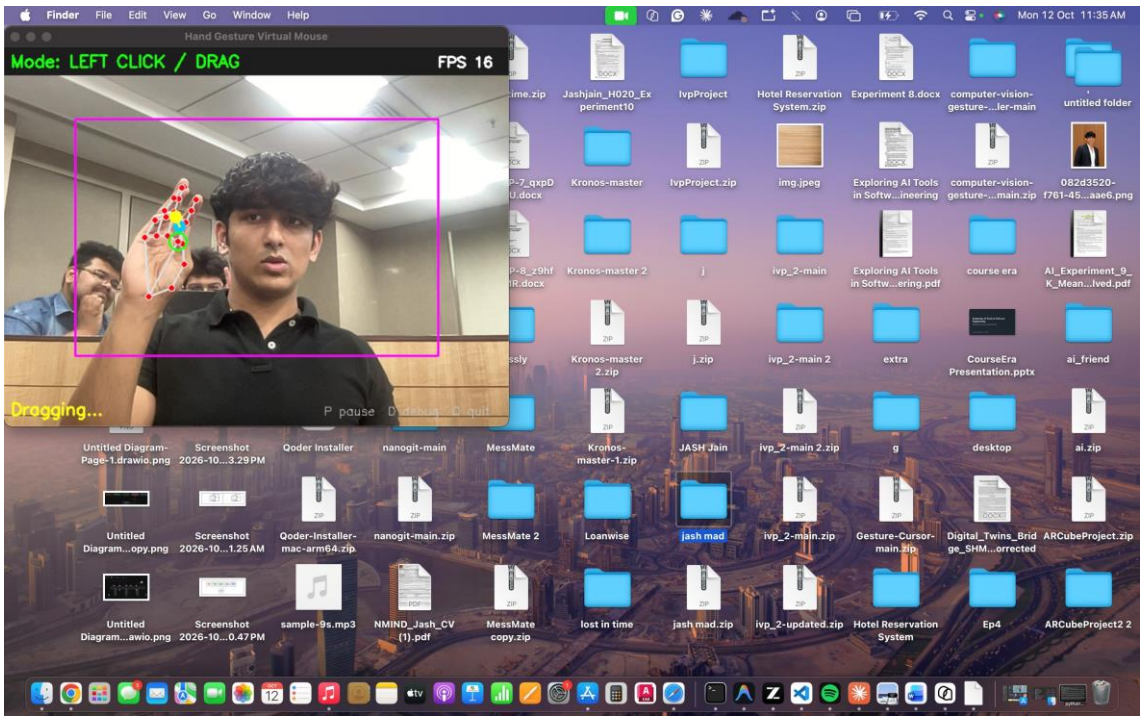


Figure 2: Active gesture recognition during testing session.

8.3 Discussion

The 1-Euro filter proved effective at balancing jitter reduction with responsiveness. During stationary positioning, cursor movement was imperceptible on a standard display. During deliberate hand sweeps across the frame, the cursor followed with acceptable lag that did not interfere with navigation tasks. The index-knuckle tracking point was a significant improvement over fingertip tracking, as pinching motions no longer caused cursor displacement.

Gesture classification was reliable when the hand was well-lit and positioned within the central portion of the active area. Edge-of-frame positioning occasionally caused partial landmark loss, leading to brief misclassification. The debounce mechanism effectively suppressed these transient errors. The hysteresis thresholds for finger extension and pinch detection eliminated the rapid state toggling that was observed in early prototypes using single-threshold classification.

Drag-and-drop required some user adaptation to the 0.4-second hold threshold. Users accustomed to quick pinch-release motions occasionally triggered unintended drags. Adjusting DRAG_HOLD_TIME upward improved reliability at the cost of slightly slower drag initiation. The shared cooldown between right-click, double-click, and screenshot prevented accidental repeated triggers but introduced a noticeable wait when performing rapid sequential actions of different types.

9. Challenges and Limitations

9.1 Problems Encountered

- Early versions using the fingertip (landmark 8) as the pointer source exhibited severe cursor shake during pinch gestures. Switching to the index knuckle (landmark 5) resolved this but required recalibrating the mapping offsets.
- NumPy 2.x introduced breaking changes incompatible with MediaPipe 0.10.14, requiring a version pin of numpy<2 in the dependency specification.
- macOS Accessibility permissions for PyAutoGUI are not granted by default and must be manually enabled in System Settings, which was a source of confusion during initial setup.

9.2 Difficult Cases

- Backlit conditions where the hand appears as a silhouette against a bright background cause frequent detection failures. The MediaPipe model relies on visible skin texture and contrast for reliable keypoint localization.
- Rapid hand movements exceeding approximately 30 cm/s cause temporary tracking loss due to motion blur in the webcam feed at 640x480 resolution.
- Partially occluded hands (e.g., fingers hidden behind objects or other fingers) produce inaccurate landmark positions that can trigger false gesture classifications.

9.3 Computational Limitations

- All processing is CPU-bound. On lower-end hardware, frame rates may drop below 15 FPS, making the system feel sluggish and increasing gesture misclassification due to larger inter-frame displacements.
- No GPU acceleration path is implemented. MediaPipe supports GPU delegates on mobile platforms, but the desktop Python API used here does not expose this capability.

9.4 Functional Limitations

- Single-hand operation only; two-handed gestures are not supported.
- Fixed frontal camera assumption; extreme viewing angles degrade world-landmark accuracy.
- PyAutoGUI cannot control the cursor over elevated/admin windows on some operating systems.
- No persistent user profile or calibration storage; thresholds must be tuned in code.

10. Conclusion

This project implemented a complete hand gesture-based virtual mouse system that translates real-time webcam input into standard computer mouse operations. The system uses MediaPipe Hands for skeletal tracking, angle-based finger extension analysis with hysteresis for gesture classification, scale-invariant pinch detection for click actions, and the 1-Euro adaptive filter for smooth cursor control.

Seven distinct gestures were successfully mapped to mouse functions: cursor movement, left click, drag-and-drop, right click, double click, scrolling, and screenshot capture. Testing under typical indoor conditions demonstrated frame rates of 19-29 FPS with stable cursor behaviour and reliable gesture recognition when adequate lighting and clear hand visibility were maintained.

The key learning from this project is that practical touchless interaction is achievable with commodity hardware and open-source computer vision libraries, provided that careful attention is paid to signal filtering, state management, and the distinction between screen-space and world-space coordinate systems. The choice of tracking point (knuckle over fingertip), the use of hysteresis in classification thresholds, and the adaptive nature of the 1-Euro filter were all decisions that significantly impacted usability and would not have been obvious without iterative testing and refinement.

Future work could address the current limitations through multi-hand support, GPU-accelerated inference, adaptive exposure preprocessing for low-light robustness, and a calibration interface that allows per-user threshold tuning without code modification.

References

[1] Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenko, A., Sung, C., Chang, Z., ... & Grishchenko, I. (2020). MediaPipe Hands: On-device Real-time Hand Tracking. arXiv:2006.10214.

[2] Casiez, G., Roussel, N., & Vogel, D. (2012). 1-Euro Filter: An Adaptive Smoothing Algorithm for Interactive Systems. Proceedingsof the SIGCHI Conference on Human Factors in Computing Systems, pp. 2527-2530.

[3] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.

[4] Al Sweigart, A. (2023). PyAutoGUI: Cross-platform GUI automation for human beings. https://pyautogui.readthedocs.io/[5] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585, 357-362.

[6] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., George, E., Leedham, T., ... & Grundmann, M. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172.